相依架構抽取：微服務系統的 Pattern Language*

相依架構抽取

Dependency Architecture Extraction: A Pattern Language for Microservices

微服務系統的相依圖（哪個服務呼叫哪個服務、用了哪個資料庫、連到哪個外部主機）是部署排序、變更影響分析與測試規劃的基礎，但它很少被完整地記錄下來。重建它的第一步該怎麼做，學界與業界的看法已經相當一致：讀部署描述檔、從程式碼抽取呼叫點、從服務網格或分散式追蹤看實際發生的呼叫。難的是之後的事：幾個來源對同一條關係說法不一、名稱對不上、圖與報告各說各話、LLM 撰寫的報告重述事實時出錯、沒收集的資料被當成零。學界與業界處理這些問題時，有四個反覆出現的做法；本文把它們寫成一套 pattern language，命名為「相依架構抽取」：混合重建、實測優先於推測、請人確認名稱對應，以及基於事實的解釋。納入標準是兩個條件同時成立：有已發表的論文描述它，而且有業界工具採用它並寫在官方文件裡。每個 pattern 的 Known Uses 都分成論文與業界工具兩類列出，讓讀者自行判斷它的通用程度。

Abstract. The dependency map of a microservice system—which service calls which, which database each one uses, and which external hosts it reaches—underpins deployment ordering, change impact analysis and test planning, yet it is rarely documented in full. Research and industry already agree on where recovering it starts: read the deployment descriptors, extract call sites from the code, and observe actual calls through a service mesh or distributed tracing. The hard part comes next: sources disagree about the same relation, names do not match, the map and the report contradict each other, LLM-written reports restate facts wrongly, and data that was never collected is shown as zero. This paper writes up four practices that research and industry use for these problems as a pattern language, Dependency Architecture Extraction: Hybrid Reconstruction, Observation over Inference, Human-Confirmed Name-Mapping, and Fact-Based Explanation. A practice is included only if it is described in published research and adopted by industrial tools in their official documentation. Each pattern lists its known uses from research and industry, so readers can judge its generality themselves.

CCS CONCEPTS • Software and its engineering → Software reverse engineering; Software architectures; Patterns

Additional Keywords and Phrases: pattern language、微服務、架構抽取、架構重建、相依圖、服務網格、分散式追蹤、human-in-the-loop、大型語言模型

ACM Reference Format:
李安喻. 2027. 相依架構抽取：微服務系統的 Pattern Language（Dependency Architecture Extraction: A Pattern Language for Microservices）. In HILLSIDE 13th Proc. of Asian Conf. on Pattern Lang. of Programs, People, and Practices (AsianPLoP). 26-29 (March), Keelung, Taiwan, 19 pages.

1 緒論

1.1 情境

你負責一個由幾十個服務組成的系統，或者你在做一個工具，要替這種系統自動畫出**相依圖**：服務之間誰呼叫誰、哪個服務用了哪個資料庫或佇列、哪些呼叫連到叢集外。這張圖會被拿來做決定：部署順序怎麼排、改一個服務要通知誰、測試還缺哪裡。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission. A preliminary version of this paper was presented in a writers' workshop at the 13th Asian Conference on Pattern Languages of Programs, People & Practices (AsianPLoP).
© 2027
AsianPLoP'27, March 26–29, Keelung, Taiwan. Copyright 2027 is held by the author(s). ISBN 〔TBA〕

沒有人能直接給你這張圖。架構文件在第一次發布後就開始過時；程式碼散在幾十個 repo、好幾種語言裡；真正跑起來的流量又只走過其中一部分路徑。所以你得從好幾個地方拼出它，而每個地方看得到的東西都不一樣（表 1）。

表 1：各個來源看得到與看不到的東西

來源	範例	看得到	看不到
部署描述檔	Kubernetes manifest、Docker Compose、Helm chart	有哪些服務；設定上誰連到誰（wiring，例如環境變數裡的位址、depends_on）	程式碼裡真的有沒有呼叫
程式碼與設定檔	HTTP client、Feign、gRPC stub、資料庫連線設定	寫在那裡的呼叫，不管有沒有跑到	動態分派、沒有解析器的語言、宣告了卻沒用到的設定
執行期觀測	服務網格的呼叫計數、分散式追蹤	這段期間真的發生過的呼叫	沒被走到的路徑
人	熟悉系統的工程師	名稱的對應、只有負責部署的人才知道的值（例如帳密、帳號）	規模大時問不完

「執行期觀測」是什麼。系統跑起來之後，每個服務旁邊會掛一個 proxy（例如 Istio 服務網格的 sidecar），或者程式裡加入了追蹤用的函式庫（例如 OpenTelemetry）。服務之間的每一次呼叫都會被它記下「誰呼叫了誰、幾次、成功與否」，送到 Prometheus 這類時間序列資料庫或追蹤系統。這類資料在監控領域叫 telemetry，架構重建（architecture recovery）的文獻把靠它的方法叫動態分析。它是呼叫真的發生時被記下來的，所以精確；但只有真的被呼叫過的關係才會出現。

1.2 問題

來源有好幾個、可靠度各不相同，要怎麼拼出一張讀者能拿來做決策的圖？

已有研究量化過這個問題的難度。一份比較研究找到 13 個靜態架構重建工具、實際比較其中 9 個，表現最好的單一工具 F1 是 0.86，依特徵選出最佳組合（共四個工具）後是 0.91 [Schneider et al. 2025]；37 個重建工具中，19 個純靜態、10 個純動態、只有 8 個兩者都用 [Bakhtin et al. 2024]；2026 年的一份文獻回顧直接寫「沒有任何單一來源足夠」，並把「可靠地融合異質證據」列為未解的問題 [Grinenko et al. 2026]。本文整理學界與業界反覆使用的做法，寫成 pattern：每個做法解什麼問題、要在哪些 forces 之間取捨、有什麼代價，以及做法之間如何銜接。這些 pattern 是從已發表的研究與業界工具的官方文件中歸納出來的（第 1.3 節的兩個條件）。

1.3 納入標準：兩個條件

PLoP 系列沿用 Christopher Alexander 的定義：pattern 是針對某個情境中反覆出現的問題所提出的解法，這個解法要在互相拉扯的 forces 之間取得平衡。其中「反覆出現」指的是：pattern 是社群已經在用的做法，不是某一個人的發明。本文把「反覆出現」具體化成兩個同時成立的條件（表 2），缺一就不納入；第 9 節列出所有候選的篩選結果，包括沒有通過的，以及通過了但被併進其他 pattern 的。

表 2：本文的納入標準

條件	要求	為什麼
論文	至少兩篇已發表的論文描述或評估這個做法	證明它被研究過，而且已知它的限制
業界	至少兩個業界工具採用這個做法，並寫在官方文件裡	表示真實產品已經在用，不只存在於論文裡

兩個條件之外，還有一個區分「pattern」與「步驟」的判準：一個做法要有自己互相拉扯的 forces，讓人需要在幾種做法之間取捨，才寫成 pattern；只有一種顯而易見的做法的，寫成背景（第 2 節）或某個 pattern 的前提。

每個 pattern 自成一節（第 5 至 8 節），依同一組欄位撰寫。節的開頭是名稱（優先沿用業界或論文已在用的叫法）、一句 slogan、別名與一句話摘要；之後每個欄位一個小節：Context、Problem、Forces、Solution（附流程圖與商店例子）、實作要點、Resulting Context、Consequences、Known Uses、Related Patterns。讀者是做「從多個來源拼出一張系統圖」的人，不限微服務：相依分析工具的開發者、服務目錄與平台工程團隊、可觀測性平台的開發者，以及要把分析結果交給 LLM 寫成報告的系統開發者。

2 背景：三個證據來源

重建相依圖的第一步該怎麼做，學界與業界已經相當一致，所以本文把這三個做法當成背景，不寫成 pattern。

讀部署描述檔。Kubernetes manifest 與 Docker Compose 是給機器讀的，格式固定，而且部署工具每次都真的讀它們，所以不太會過時。它們提供服務清單，以及環境變數裡的位址、depends_on 這類連線設定；MicroDepGraph 就從 docker-compose 中定義的相依關係（再加上 Java 原始碼中的內部 API 呼叫）取得邊 [Rahman et al. 2019]，Soldani 等人則從 Kubernetes manifest 取得節點 [Soldani et al. 2022]；Backstage 的服務目錄也以 dependsOn 宣告相依關係 [Backstage 2026b]。它的限制是：設定了連線不等於真的呼叫。

抽取程式碼呼叫點。比對 REST client 呼叫、持久化 annotation（如 @Entity）等程式碼樣式 [Bushong et al. 2022]，或以關鍵字搜尋原始碼與設定檔，讓每個模型元素都能追溯到程式碼片段 [Schneider and Scandariato 2023]。它涵蓋的範圍廣，但看不到動態分派與沒有解析器的語言，而且寫在那裡的呼叫不一定會被執行。

看執行期服務圖。從服務網格的呼叫計數或分散式追蹤，畫出這段期間真的發生過的呼叫 [Al Maruf et al. 2022]；Kiali、Grafana Tempo、Datadog 與 AWS X-Ray 的服務圖都是這樣產生的 [Kiali 2026; Grafana Labs 2026b; Datadog 2026c; Amazon Web Services 2026a]。它最精確，但只看得到被走過的路徑。

這三個做法都通過本文的兩個條件，但我們沒有把它們寫成 pattern：它們各自只有一個顯而易見的做法，沒有真正互相拉扯的 forces，而且第 1.1 節的表 1 已經說完了它們的取捨。本文的四個 pattern 處理的是三個來源都有了之後的問題。

3 貫穿全文的例子

全文用同一個線上商店當例子（圖 1）。web 是入口，orders 處理訂單並呼叫付款與出貨，catalog 與 orders 各有自己的資料庫，cart 是購物車。

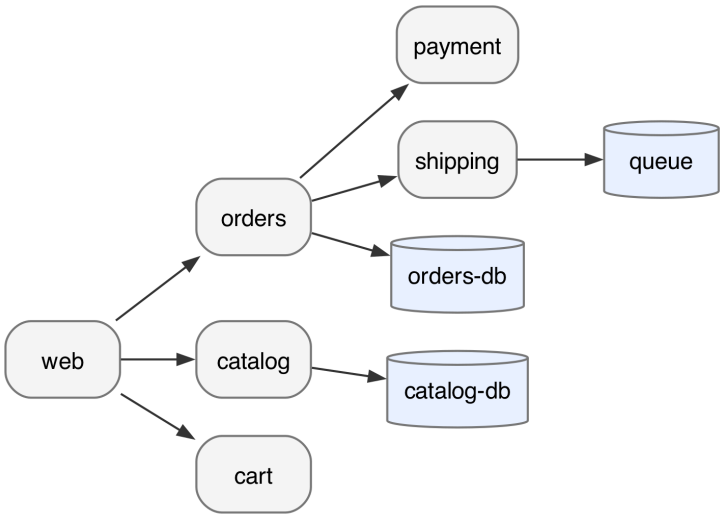


圖 1：線上商店實際的相依關係（作為全文的例子）

表 3 是同一組關係在各個來源裡的樣子。除了 web → orders 與 orders → orders-db，其餘每一列都至少有一個來源說錯或沒說，後面的 pattern 會逐一處理。

表 3：線上商店的各條關係，在各個來源裡長什麼樣子

關係	部署描述檔	程式碼	執行期觀測	文件
web → orders	ORDERS_ADDR=orders:8080	OrderClient	95 次請求	有
orders → payment	PAYMENT_ADDR	PaymentClient.java:42	這段期間沒人付款，0 次	有
orders → shipping	沒有	沒有（目標經 registry 動態解析）	12 次	有
orders → orders-db	DB_URL	JPA @Entity	12 次 TCP 連線	有
catalog → catalog-db	DB_URL	沒有任何持久化程式碼	0 次	沒有
web → cart	CART_ADDR	以 Node 撰寫，沒有解析器	沒有資料（cart 沒有 sidecar，這次收集不到）	有，但文件稱它 basket-service

4 PATTERN LANGUAGE 全貌

四個 pattern 分成兩組（圖 2）。**建圖**這組（①–③）回答「三個來源都有了，怎麼拼出一張可信的圖」；**交付**這組（④）回答「把圖變成數字、報告與問答時，怎麼不走樣」。箭頭上的字，是前一個 pattern 套用後留下的問題，由下一個 pattern 接手處理。相鄰的 pattern 就這樣一個引出下一個；pattern 之間彼此銜接，使整體具有 Alexander 所說的 generative 性質，這也是本文稱它為 language、而不是 catalog 的原因。

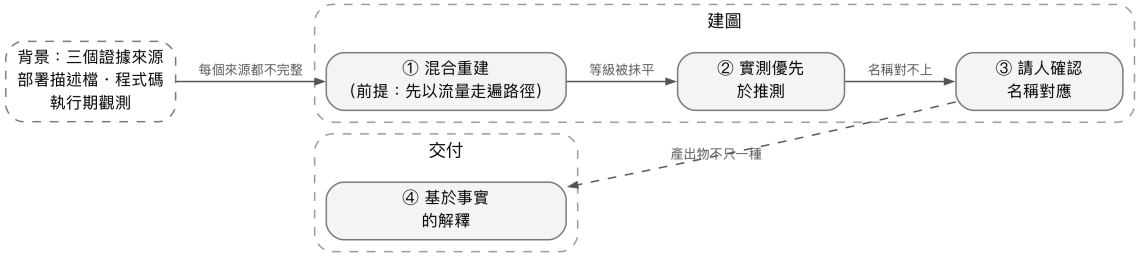


圖 2：「相依架構抽取」pattern language 全貌

表 4 是四個 pattern 的摘要：每個 pattern 解決的問題與解法，各用一句話說完。第 5 至 8 節依序是 ①–④，每節開頭有一句 slogan，Solution 小節附一張流程圖（圖 3 至圖 6）。

表 4：四個 pattern 的摘要

Pattern	Problem	Solution
① 混合重建（Hybrid Reconstruction）	每個來源都不完整，看不到的地方各不相同	先以流量把執行期觀測補齊，再以「來源 → 目標」為鍵合併成一張圖，每條邊保留它來自哪些來源
② 實測優先於推測（Observation over Inference）	全部畫成一樣，讀者分不出哪條可信	每條邊帶「實測／推測」標記並畫成不同樣子，更強的證據取代較弱的
③ 請人確認名稱對應（Human-Confirmed Name-Mapping）	規則判斷不了的名稱，丟掉或模糊比對都會出錯	把它變成附候選的具體問題交給熟悉系統的人，答案存下來
④ 基於事實的解釋（Fact-Based Explanation）	各產出物各自推導而互相矛盾；LLM 重述事實會錯；沒收集的資料被當成 0	先建一份統一模型，事實由程式從它產生，LLM 只寫詮釋與建議；沒收集的寫「未量測」，不寫 0

照順序從頭讀，對應的是一個工具從零開始建構的順序。已經有一張圖、正要把它接到 LLM 產生報告的團隊，可以直接讀 ④；只做靜態分析、沒有執行期觀測的團隊會最先遇到 ④ 最後一條規則處理的問題，因為所有執行期欄位都是空的。

5 混合重建（HYBRID RECONSTRUCTION）

Slogan：一個來源看不全，就讓它們互補。（No single source sees it all.）

別名：靜態＋動態分析、多來源融合（information fusion）；ServiceNow 稱為 pattern-based 與 traffic-based mapping；vFunction 稱為 dynamic 與 static analysis。

一句話摘要：先主動產生流量，讓執行期觀測看到平常沒被走到的路徑；再把連線設定、程式碼、執行期觀測與文件的結果，以「來源服務 → 目標服務」為鍵，合併成同一張圖，每條邊保留它來自哪些來源。

5.1 Context

你有好幾個來源，每一個都看到一部分。讀者只想看一張圖。

前提：先以流量走遍路徑。三個來源裡執行期觀測最可信，但它只有被走過的路徑：在測試環境裡可能根本沒有真實使用者，在正式環境裡罕見流程（退款、管理功能）可能一週才走一次。所以在合併之前，先用負載

產生器、端對端測試或合成監控腳本主動產生流量，優先走主要的使用者流程；把靜態來源畫出、但執行期還沒量測到的邊列成清單，用它決定下一輪要送哪些請求；清單不再縮短時就停。動態分析本質上不完整，難處在於決定要執行哪些情境 [Cornelissen et al. 2009]；Istio 的 Kiali 教學要使用者先反覆送流量（多次 curl，或 watch 每秒一次），因為圖呈現的是一段時間內流經網格的流量 [Istio 2026]；Online Boutique 這類範例系統內建負載產生器，持續送出模仿真實購物流程的請求 [Google Cloud Platform 2026]。這一步只有一種顯而易見的做法，所以本文把它寫成前提而不是 pattern；但沒有它，執行期來源會缺一大塊，後面的合併就是在一副不完整的骨架上疊東西。三件事要注意：流量要語意正確才走得到深處（登入要對的帳密、轉帳要一個存在的收款帳號），需要只有人知道的值時請熟悉系統的人提供（→ ③），不要編一個；寫入類的流量要在測試環境跑，或使用可以還原的資料；判斷「走到了沒」要看相依的邊有沒有出現，而不是只看 HTTP 狀態碼，因為有些應用拒絕表單時也回 HTTP 200。

5.2 Problem

每個來源都不完整，而且看不到的地方各不相同。只用一個，一定漏；全部合在一起，又要決定名稱不一致與說法衝突時聽誰的。

5.3 Forces

- **來源之間確實互補。**靜態看不到動態分派，執行期看不到沒走過的路徑；有些關係只在執行期才看得到 [Mayer and Weinreich 2018]，而依特徵挑選四個靜態工具組合起來，F1 就從 0.86 提高到 0.91 [Schneider et al. 2025]。
- **名稱不一致。**同一個服務在部署描述檔、程式碼、文件裡可能有三種寫法。
- **合併規則要能解釋。**加權投票、機率模型很難向讀者說明一條邊為什麼在圖上。
- **弱來源會帶進雜訊。**文件的錯誤、宣告了卻沒用到的設定，合進來就成了假邊。

5.4 Solution

因此：

1. 先確定節點：以部署描述檔的服務清單為主，其他來源的名稱對齊到它；對不上的另外處理（→ ③）。
2. 以（來源服務，目標服務）為鍵合併各來源的邊。
3. 每條邊保留**所有**支持它的來源，以及出處（provenance：檔案與行號、指標名稱、文件段落）。
4. 較弱的來源只**補強**既有的節點，不憑空新增部署裡不存在的服務。
5. 記錄哪些來源這次有收集、哪些沒有（→ ④）。

圖 3 把這些步驟畫成流程。

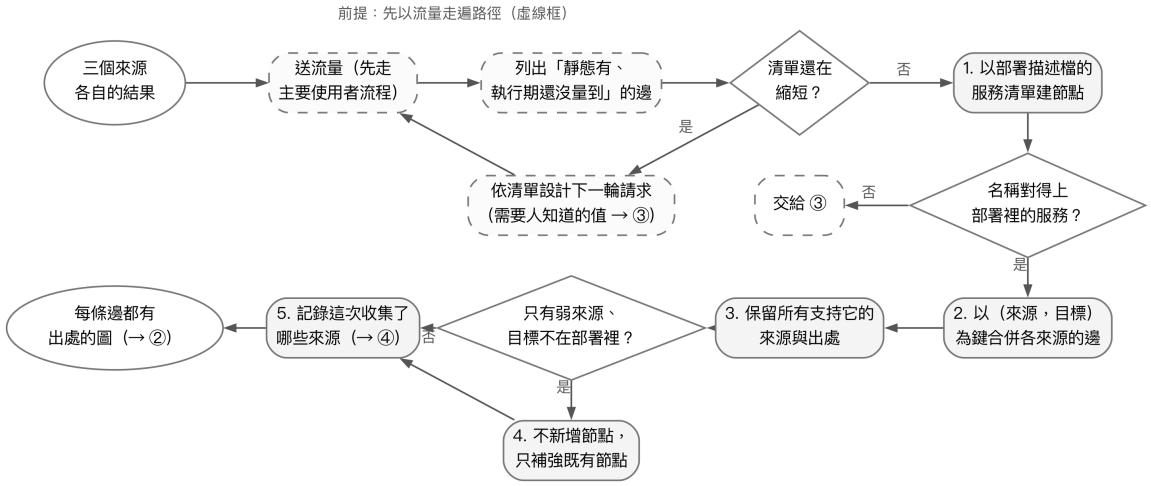


圖 3：混合重建的 Solution 流程（虛線框是前提；編號是 Solution 的步驟）

例如：靜態來源說有 orders → payment，執行期是 0 次；下一輪流量加一筆結帳之後，orders → payment 就出現了。合併後，orders → shipping 來自執行期與文件；orders → payment 來自連線設定、程式碼與執行期；catalog → catalog-db 只來自連線設定。

5.5 實作要點

合併的順序會影響結果的可解釋性：先以最可靠的來源建骨架，再依序疊上較弱的來源，每一步只做加法。框架函式庫的名稱（例如快取、斷路器 circuit breaker 的函式庫）常被誤認成服務，要用一份可設定的排除清單把它們濾掉。流量腳本不必從零寫：repo 裡常常已經有範例專案附的負載產生器、整合測試、README 裡的 curl 指令；腳本用標準格式（例如 Postman collection）存下來，下一次可以重跑。

5.6 Resulting Context

你有一張比任何單一來源都完整的圖，每條邊都知道自己從哪裡來。但畫出來的時候，這幾條邊長得一模一樣：讀者分不出 web → orders（量測到 95 次）和 catalog → catalog-db（只有一個連線設定）哪條可信（→ ②）。

5.7 Consequences

- ＋ 完整度比單一來源高，這一點有量化研究支持。
- ＋ 出處保留下來，每條邊都可以回頭查證；「靜態有、執行期還沒量測到」的邊清單本身就是一份測試缺口清單。
- － 需要名稱對齊；對齊錯了會合錯。
- － 弱來源的雜訊會跟著進來，需要 ② 讓讀者分得出來。
- － 要維護流量腳本；寫入類的流量需要測試環境，而且流量永遠不會走到 100%，需要停止條件。

5.8 Known Uses

表 5：混合重建：Known Uses

類別	誰	怎麼做
論文	Mayer 與 Weinreich [Mayer and Weinreich 2018]	有些關係只在執行期才看得到，所以把靜態的服務資訊，和基礎設施與執行期的資訊結合起來
論文	Schneider 等人 [Schneider et al. 2025]	表現最好的單一靜態工具 F1 0.86，依特徵選出四個靜態工具的最佳組合後 0.91
論文	Grinenko 等人 [Grinenko et al. 2026]	指出沒有單一來源足夠，而動態重建的品質取決於 workload coverage 與所用資料是否具代表性
業界	ServiceNow Service Mapping [ServiceNow 2026b]	以 traffic-based discovery 補充 pattern-based 的對應，再把連線資訊補進服務圖
業界	vFunction [vFunction 2026]	把靜態分析的結果疊在動態分析之上

5.9 Related Patterns

下一個是 ② 實測優先於推測。名稱對不上時用 ③；前提裡的流量需要只有人知道的值時，也用 ③ 的方式請人提供。

6 實測優先於推測（OBSERVATION OVER INFERENCE）

Slogan：量到的畫實線，猜的畫虛線。（Draw what you measured solid, what you inferred dashed.）

別名：推測節點（inferred services，Datadog [Datadog 2026b]）、推測區段（inferred segments，AWS X-Ray）、虛擬節點（virtual nodes，Grafana Tempo [Grafana Labs 2026b] 與 OpenTelemetry）；學界稱為 uncertainty in graph edges、partial models。

一句話摘要：讓每條邊或節點帶一個「實測」或「推測」標記並畫成不同樣子；同一條邊有了更強的證據，就取代推測的版本。

6.1 Context

你已經合併出一張多來源的圖（①），每條邊都有出處。讀者會拿它做決策，而且通常不會回頭看出處。

6.2 Problem

全部畫成一樣，讀者會以為每條都一樣真；只畫量測到的，又會丟掉量測不到但確實存在的相依關係。

6.3 Forces

- **可靠度和完整度此消彼長。**實測的可靠但不完整；推測的涵蓋較廣，但可靠度較低。
- **讀者要一眼看懂。**分太多級看不懂，只有一級又什麼都分不出來。
- **同一條邊會被多個來源提到。**不能畫成平行的好幾條，也不能讓弱的蓋掉強的。
- **視覺通道有限。**線型、顏色、粗細要分給可信程度、關係類型與流量大小；研究在邊上同時編碼主屬性與不確定性，發現部分視覺變數組合（例如色相配明度）干擾明顯，不同組合的差異很大 [Guo et al. 2015]。
- **誠實可能降低信任。**標出「這是推測的」，讀者可能覺得整張圖都不可靠。

6.4 Solution

因此：

1. 給每條邊（或節點）一個標記：**實測**（執行期觀測記錄到）或**推測**（只來自程式碼、設定或文件）。
2. 標記和出處分開儲存：一條邊可以有多筆出處，但只有一個標記。
3. 合併時**更強的證據取代較弱的**：同一條邊一旦被量測到，就不再顯示為推測。
4. 把最醒目的視覺通道（實線與虛線，或實框與虛框）給這個標記；關係類型用顏色或短標籤，流量大小用線寬或不畫。
5. 下游（摘要、報告）只能照抄或降級，不能把推測的講成確定的。

圖 4 把這些步驟畫成流程。

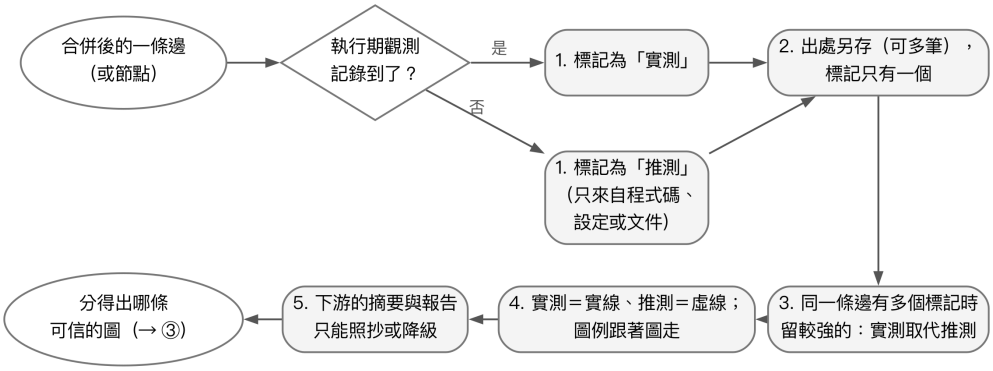


圖 4：實測優先於推測的 Solution 流程

例如：web → orders 畫實線；orders → payment 與 catalog → catalog-db 畫虛線。下一輪流量量測到 orders → payment 之後，它變成實線。

6.5 實作要點

標記要放在圖例裡，而且圖例要跟著圖走（匯出成圖片時也在）。標記的變化本身是有用的訊號：一條推測的邊變成實測，代表流量終於走到了。兩級是底線；如果要再細分（例如把「程式碼裡有呼叫」和「只有連線設定」分開），每多一級都要在圖例裡交代，而且讀者得學會分辨。

6.6 Resulting Context

讀者分得出哪條可以據以決策。但合併時有些名稱根本對不上：文件裡的 basket-service 是不是 cart？規則判斷不了，標記也幫不上忙，因為它還沒有被合進任何一條邊（→ ③）。

6.7 Consequences

- ＋ 量測不到、但確實存在的相依關係還在圖上，而且不會被誤認為已確認。
- ＋「取代」的規則簡單，不需要調參數。
- － 讀者要學會看圖例。
- － 推測對了的邊和推測錯了（其實不存在）的邊，在圖上都只是一條虛線；要分清楚還得回頭看出處。

6.8 Known Uses

表 6：實測優先於推測：Known Uses

類別	誰	怎麼做
論文	Guo 等人 [Guo et al. 2015]	以受控實驗比較在圖的邊上呈現不確定性的視覺變數
論文	Famelis 等人 [Famelis et al. 2012]	主張模型應能表達建模者的不確定性，過早選定一個答案會帶進錯誤資訊
業界	AWS X-Ray [Amazon Web Services 2026a]	以下游呼叫產生 inferred segment，該服務自己送出資料後就取代它
業界	AWS CloudWatch Application Map [Amazon Web Services 2026b]	沒有被量測的服務以虛線框或不同顏色呈現
業界	Datadog [Datadog 2026b]	從呼叫端的資料推測出沒有安裝追蹤的服務，標為 inferred service

6.9 Related Patterns

前一個是 ①；下一個是 ③ 請人確認名稱對應。容易和 ④ 的「未知」混淆：這個 pattern 講「多可信」，④ 的最後一條規則講「有沒有查過」；一條推測的邊是有弱證據的，一個「部署狀態未知」的節點是完全沒查過的，兩者不能共用同一個欄位。

7 請人確認名稱對應（HUMAN-CONFIRMED NAME-MAPPING）

Slogan：規則判斷不了的名稱，問人，不要猜。（When the rules cannot tell, ask; do not guess.）

別名：Reflexion model 的 mapping；互動式重複資料比對（interactive deduplication）；Datadog 稱為 service remapping；Dynatrace 稱為 merged services 與 service detection rules；ServiceNow 稱為手動加入與排除連線。

一句話摘要：規則判斷不了某個名稱該對應到誰時，不要丟掉、也不要模糊比對；把它變成一個具體的問題（附候選）交給熟悉系統的人，答案存下來給下一次用。

7.1 Context

你已經合併出有標記的圖（②），但有些東西合不進去：文件說 basket-service，圖上有 cart；程式碼裡的短名稱 user 同時符合 user-service 和 user-db；某個名稱在函式庫清單裡，但部署描述檔裡它是一個真的服務。另外，① 的前提要觸發某條路徑時，可能需要一組只有負責部署的人才知道的值。

7.2 Problem

規則判斷不了的對應，自動處理只有兩條路：丟掉（藏起一條可能真的相依關係），或模糊比對（憑相似度捏造一條邊）。兩條都可能出錯，而熟悉系統的人往往知道答案。

7.3 Forces

- **人知道答案，但人的時間很貴。**一次問幾十題，很少有人會回答完。
- **人也會答錯**，而答錯的對應會被合進圖裡。
- **名稱會重複出現。**同一個專案下一次分析，同樣的名稱又會對不上。
- **規模。**大系統的名稱問不完，只能挑規則真的判斷不了的問。

7.4 Solution

因此：

1. 先用所有規則對齊；只有規則真的判斷不了的才記成問題，不默默丟掉。
2. 每個問題附上**排序過的候選**（排序只給人看，不替人做決定），以及「新節點」「忽略」兩個選項。
3. 在使用者平常就會用的地方問（審查頁面、對話），而且一次把問題問完，不要一題一題打斷人。
4. 答案以確定性的規則解析；看不懂就保留問題，不要猜。
5. 答案以 repo 為單位**存下來**，下一次只問新出現的名稱；答錯要能清除。
6. 報告裡標出哪些邊是靠人的回答畫上去的。

圖 5 把這些步驟畫成流程。

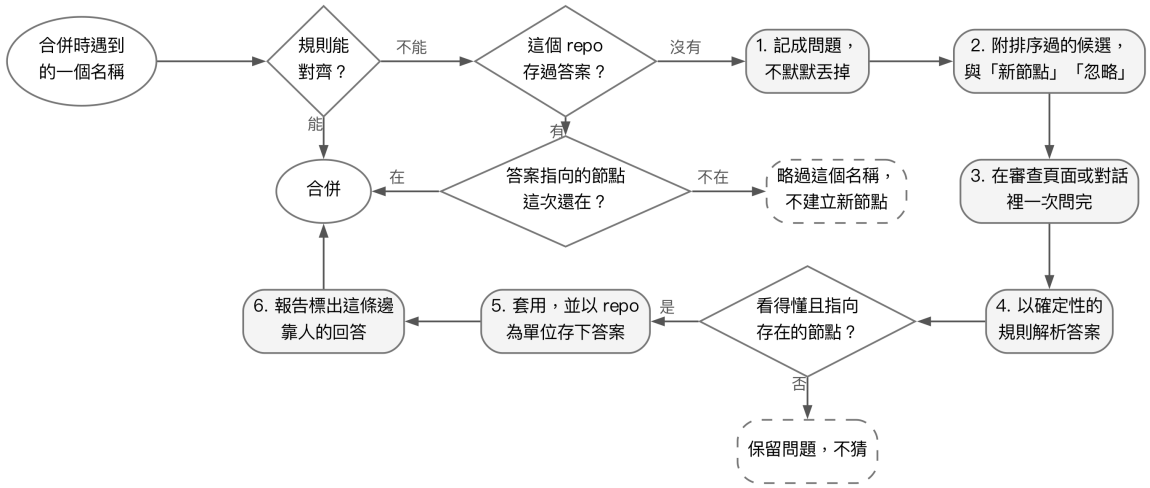


圖 5：請人確認名稱對應的 Solution 流程

例如：「basket-service 是哪一個？1 cart　2 catalog　／new　／ignore」，人答 1，文件那條邊就合到 cart 上；下一次分析不再問這個名稱。

7.5 實作要點

候選可以用名稱的包含關係、共用的詞與編輯距離排序，但排序結果只用來呈現。填入的答案若指向圖上不存在的節點，就當作沒答、保留問題；存下來的舊答案若指向這次圖上不存在的節點，就略過該名稱、不建立新節點，以免系統改名之後舊答案憑空造出節點。

7.6 Resulting Context

規則判斷不了的地方由人補上了，而且補過的不會再問。現在你有一張整理過、有標記的圖。接下來要交付的不只是圖：還有覆蓋率之類的數字、一份報告，之後可能還有人追問（→ ④）。

7.7 Consequences

- ＋ 不再默默丟掉相依關係，也不憑相似度捏造相依關係。
- ＋ 答案累積之後，同一個專案越問越少。
- － 取決於回答的人熟不熟悉系統；答錯會進到圖裡。

- — 要有地方存答案，也要能清除。

7.8 Known Uses

表 7：請人確認名稱對應：Known Uses

類別	誰	怎麼做
論文	Murphy 等人 [Murphy et al. 1995; Murphy et al. 2001]	工程師定義高階模型並指定它如何對應到原始碼，工具據此算出模型與程式碼一致和不一致之處（FSE 1995 原始提出，TSE 2001 期刊版）
論文	Sarawagi 與 Bhamidipaty [Sarawagi and Bhamidipaty 2002]	互動式實體對應：挑出最難判斷的配對請使用者標記
業界	ServiceNow Service Mapping [ServiceNow 2026a]	使用者決定要加入或移除哪些連線，決定記錄為手動加入或手動排除
業界	Datadog [Datadog 2026d]	以 service remapping 規則改名、合併或拆分服務

7.9 Related Patterns

前一個是 ②；下一個是 ④ 基於事實的解釋。① 的前提需要只有人知道的值時，也用同樣的方式問。

8 基於事實的解釋（FACT-BASED EXPLANATION）

Slogan：事實由程式說，AI 只負責解釋。（Let the program state the facts; let the AI explain them.）

別名：analytics augmented generation（Satyrn）；program-aided language models（PAL）；Dynatrace 稱為 hypermodal AI（causal AI 計算、generative AI 解釋）；K8sGPT 稱為 analyzer 加上 explain。事實所出自的那份模型，業界叫 Software Catalog（Backstage）或拓樸模型（Dynatrace Smartscape），學界叫架構重建的 metamodel 或中間表示法（intermediate representation），相當於 Model–View–Controller 裡的 model；「沒有資料」的狀態，監控工具叫 No Data（Grafana、Datadog），Kubernetes 叫 Unknown condition，runtime verification 叫 inconclusive verdict。

一句話摘要：每次分析先建一份統一模型；能由程式從模型算出的事實（有哪些相依關係、量測到幾次、覆蓋率多少、哪些來源這次沒收集）由程式產生，LLM 只負責解釋這些事實、提出詮釋與建議，不讓它重述事實；沒收集到的資料在模型裡是「未知」，程式照寫「未量測」而不是 0，也不交給 LLM 猜。

8.1 Context

你有一張整理過、有標記的圖（③）。要交付的不只是圖：還有覆蓋率之類的指標、一份文字報告，之後可能有人追問細節，或有別的工具要讀。它們由不同的程式產生，有的是畫圖程式、有的是計分程式。報告你打算讓 LLM 寫，因為它能寫出讀者願意讀的通順文字：這個服務扮演什麼角色、哪裡有風險、下一步先做什麼。另外，工具不是每次都在同樣的條件下跑：有時沒有叢集、有時追蹤沒開、有時某個抽取步驟失敗了；而工具通常設計成某一步失敗也不中斷，因為讀者寧願拿到不完整的報告，也不要什麼都沒有。

8.2 Problem

圖、數字與報告各自從原始來源推導，就會說不同的話；報告交給 LLM 寫，它重述事實時會漏、會加、會換詞，把 prompt 寫得更嚴格不一定能解決；沒收集的資料在程式裡變成 0 或 false，在 LLM 手上變成猜測。三種走樣的結果一樣：讀者拿到一份看起來完整、其實不對的報告，而且無從判斷哪裡不對。

0% 看起來像「量了，很糟」；deployed: false 看起來像「查了，沒部署」；空的相依關係清單看起來像「找過了，沒有」。

8.3 Forces

- **每個產出物都想要最順手的輸入。**畫圖要結構，計分要清單，LLM 要文字；推導邏輯一重複就容易分歧，而且只在特定資料下才看得出來。
- **讀者要連貫的文字**，不想看兩種風格拼在一起；但完全不用 LLM，報告就只剩一張讀者不太願意讀的表。
- **改 prompt 的成本很低，所以很誘人**：改一次看起來好了，換一個系統又錯。
- **LLM 自己算事實的準確度不夠。**讓 LLM agent 自己重建微服務架構，受測的兩個模型中較好的 GPT-4o 平均 F1 約 0.6 [Liljas et al. 2026]。
- **詮釋需要事實。**不給 LLM 看事實，它的詮釋可能和事實矛盾。
- **空白容易被填上。**程式的預設值（0、false、空清單）會替空白補一個值，空的段落在 LLM 看來也像是漏寫了；而讀者想要一個數字：「未量測」看起來沒做完，「0%」看起來做完了，後者更容易被接受，也因此更容易誤導。
- **有些視圖很晚才需要產生。**報告發出幾天後有人追問，原始資料可能已經不在了。

8.4 Solution

因此：

1. 每次分析**先建一個統一模型**，再從它產生所有視圖（圖、數字、報告、問答、匯出）。每個視圖都完全由模型決定（是模型的函式）；衍生計算（分數、分層、排序）只實作一次；模型可以存檔與讀回，事後才產生的視圖讀同一份模型，不重新收集。
2. 「這次收集了哪些來源」當成模型的中繼資料，傳到每一個視圖；狀態欄位用三值（是、否、未知），**預設是未知**，只有在證據明確時才設成否。
3. 把報告內容分成**事實**（有唯一正確答案、模型裡都有）與**詮釋**（角色、風險、建議）。
4. 事實由程式從模型產生，措辭固定；例如量測到的是連線就寫「連線數」，不寫「請求數」。遇到未知就寫一句話：為什麼未知，以及補什麼才會知道；絕不用 0、0/N、0% 代表未量測。
5. 讓 LLM 也看得到這些事實（或它們的原始紀錄），詮釋才有依據；但事實段落以程式那一份為準，LLM 若自己也寫了，丟掉它的版本。交給 LLM 的字串裡，未量測就寫「未量測」，不要把 0 這種佔位值交給它。
6. 讓讀者看得出哪一段是程式產生的；步驟失敗時記錄失敗與原因，不要回傳空結果。

圖 6 把這些步驟畫成流程。

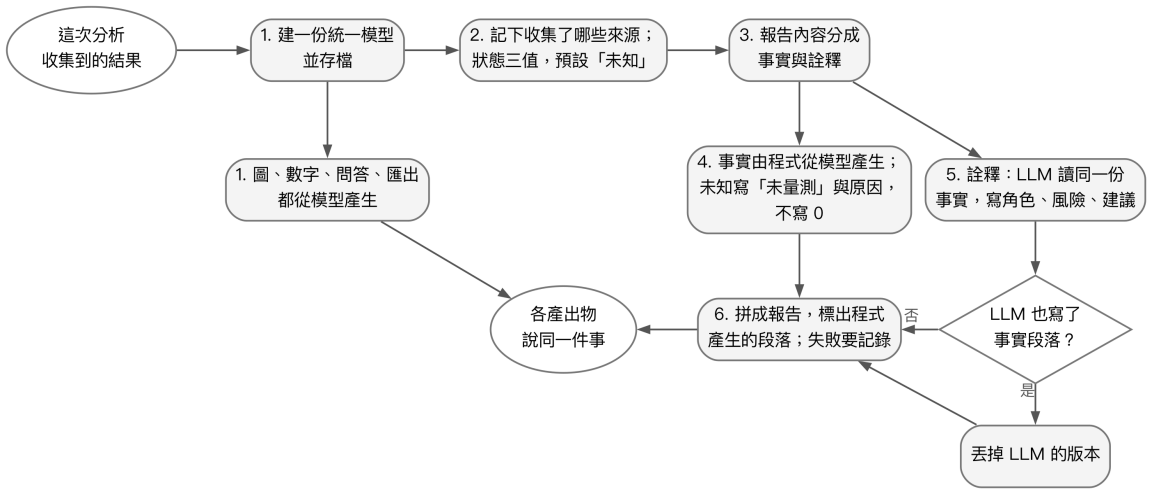


圖 6：基於事實的解釋的 Solution 流程

例如：圖上 orders → orders-db 是實線，覆蓋率把它算成已量測到，報告的「基礎設施相依關係」一節由程式產生：「orders → orders-db：量測到，12 次連線；catalog → catalog-db：只有連線設定」，三天後的問答也這樣回答，因為它們讀的是同一個物件。LLM 只寫「風險」：「catalog 和資料庫的相依關係沒有程式碼佐證，可能是殘留的設定」。這次 cart 沒有 sidecar：它的執行期欄位是「未知：這次收集不到」而不是 0 次；如果這次根本沒有連上叢集，覆蓋率顯示「未量測（連上叢集後重跑即可得到）」而不是 0/4，交給 LLM 的字串是 NOT MEASURED，而不是 0%。

8.5 實作要點

視圖完全由模型決定，所以視圖的測試不需要叢集、不需要網路，只要一個手寫的模型；「報告與圖是否一致」因此可以變成單元測試。「丟掉 LLM 的版本」比「請它不要寫」可靠，因為 LLM 不一定會照 prompt 的要求做；程式產生的段落以結構錨點（例如下一節的標題）拼進報告，找不到錨點時就附在最後，確保事實不會因為格式不如預期而遺失。三值最簡單的實作是可為 null 的布林值（nullable boolean）；比型別更重要的是預設值，預設必須是未知。「沒收集不等於零」這條規則要在每一條路徑上實施：只要有一條路徑（例如某個 prompt）沒帶「這次沒收集什麼」，空白就會從那裡被填回來。

8.6 Resulting Context

這是這個 language 的終點。讀者看得到每條邊從哪裡來（①）、有多可信（②）；名稱對不上的由人確認過（③）；每個產出物說同一件事、事實不經 LLM 重述、沒量測到的明說沒量測到（④）。套用之後會出現另一組問題，留作後續：讀者拿到報告後會追問細節，自然語言的問題怎麼變成對模型的確定性查詢。

8.7 Consequences

- ＋ 產出物之間從架構上就不會矛盾，不必靠事後校對；事實段落保證和圖一致，不必反覆修 prompt。
- ＋ 事實錯就是模型錯，除錯方向明確；加新視圖不必重寫推導邏輯，存檔讓事後的追問有據可答。
- ＋ 讀者不會把沒發生的量測當成結果；不論只做靜態分析還是完整分析，都能交出同一套產出物，而且不會誤導讀者。

- － schema 是所有視圖的共同介面，改動要照顧所有使用者；模型沒有收進來的資訊，所有視圖都拿不到。
- － 報告有一段是範本文字，風格不完全一致；事實與詮釋的界線要人劃，劃錯了 LLM 會在詮釋裡重述事實。
- － 每一個讀取點都要處理三值。

8.8 Known Uses

表 8：基於事實的解釋：Known Uses

類別	誰	怎麼做
論文	Alshuqayran 等人 [Alshuqayran et al. 2018]	以 metamodel 加上對應規則，把實作產物（程式碼、docker-compose、建置檔、Dockerfile）對應到同一個架構 metamodel，再從它產生視圖
論文	Satyrn [Sterbentz et al. 2024]	先以分析程式算出事實再交給 LLM；報告中正確的陳述超過 86%，直接讓 GPT-4 Code Interpreter 撰寫是 57%
論文	PAL [Gao et al. 2023]	LLM 寫程式、由直譯器算出答案，LLM 不自己算
論文	Bauer 等人 [Bauer et al. 2011]	只觀察到部分執行軌跡時，判定結果有 true、false、inconclusive 三種，不把「還不知道」當成 false
業界	Backstage [Backstage 2026a]	集中的服務目錄記錄擁有者與中繼資料，多數工具可以圍繞目錄裡的實體來組織
業界	Dynatrace [Dynatrace 2026; Reitbauer and Greifeneder 2023]	Smartscape 是平台內建的拓樸模型，平台各處（圖表、儀表板、警示）都以同一套拓樸模型為基礎；Davis 的預測與因果 AI 提供確定性的答案，並作為 Davis CoPilot（生成式 AI）以自然語言回答時的上下文（CoPilot 的出處為官方部落格，2023）
業界	K8sGPT [K8sGPT 2026]	診斷由預先寫好的 analyzer 完成，LLM 只在解釋分析結果時使用
業界	Grafana [Grafana Labs 2026a]	警示規則有獨立的 No Data 狀態，定義為查詢成功但沒有任何資料點
業界	Datadog [Datadog 2026a]	監控設定可以選 Evaluate as zero（空結果補成 0）或 Show NO DATA 等處理方式，把「補成 0」做成要明確選的選項

8.9 Related Patterns

接在 ③ 後面。② 的「推測」和這裡的「未知」不能共用同一個欄位：推測的邊有弱證據，未知是完全沒查過。和 PLoP 2023 的 Prompt Pattern Catalog [White et al. 2023] 方向互補：那一組 pattern 教你把 prompt 寫好，這個 pattern 說有些事不該交給 prompt。

9 篩選紀錄

表 9 是所有候選做法的篩選結果，業界文件的查證日期為 2026 年 10 月 13 日。「強」表示有微服務或服務圖領域的直接證據；「中」表示證據來自相鄰領域、一般軟體工程或非正式來源；「弱」表示證據零星，或借自其他領域。

表 9：候選做法的篩選結果

候選	論文	業界	結果
讀部署描述檔	強	強	背景（第 2 節）
抽取程式碼呼叫點	強	強	背景（第 2 節）
看執行期服務圖	強	強	背景（第 2 節）
以流量走遍路徑	中（微服務專屬的少）	強	① 的前提
① 混合重建	強	強	納入
② 實測優先於推測	中（來自視覺化與建模）	中（多為兩類、標在節點）	納入
③ 請人確認名稱對應	中（一般領域）／弱（微服務）	中（多為事後寫規則）	納入
一份目錄、多個視圖	中到強（以綜述與 metamodel 為主）	強	併入 ④（Solution 第 1 步）
④ 基於事實的解釋	中（沒有軟體架構專屬的論文）	中（部落格、開源專案）	納入
無資料不等於零	弱（借自邏輯與驗證）	強	併入 ④（Solution 第 2、4 步）
覆蓋率的分母只算量測得到的項目	未找到	只有程式碼覆蓋率工具	不納入

三個候選通過了兩個條件，卻沒有單獨成為 pattern。「以流量走遍路徑」只有一種顯而易見的做法（送流量、看哪些邊還沒出現、再送），沒有需要取捨的 forces，所以寫成 ① 的前提。「一份目錄、多個視圖」和「無資料不等於零」原本各自成篇，但它們和「基於事實的解釋」解的是同一個問題（產出物不走樣）的三個面向：事實從哪裡算（同一份模型）、由誰寫（程式，不是 LLM）、算不出來時寫什麼（未知，不是 0）；拆開寫，每一篇的 Solution 都要引用另外兩篇才完整，所以併成一個 pattern，各自變成 Solution 裡的步驟。最後一列有其他領域的類比（程式碼覆蓋率工具會列出被排除的程式碼行），但在相依分析與架構重建的領域裡，我們沒有找到描述它的論文，也沒有找到在服務圖上這樣做的業界工具；依本文的標準，它是一個工具的設計選擇，而不是一個 pattern。表中標「中」或「弱」的地方，弱在哪裡都寫在括號裡；其中 ③ 最需要說明：業界的做法多半是使用者事後寫改名、合併、拆分的規則，而不是工具在分析中提出問題。ServiceNow 讓使用者審查建議的連線，是最接近本 pattern Solution 的做法。

10 相關研究

Cerny 等人整理了微服務架構重建與視覺化的技術 [Cerny et al. 2022]；Bakhtin 等人把 37 個重建工具依靜態、動態與混合分類 [Bakhtin et al. 2024]；Grinenko 等人指出沒有單一來源足夠，並把「可靠地融合異質證據」與「跨方法共用的中間表示法」列為未解的問題 [Grinenko et al. 2026]。本文的 ①②④ 對應的是學界與業界目前處理這兩個問題的部分做法；本文把它們寫成可以照著做的形式，並說明各自的代價。

在 AsianPLoP、PLoP 與 EuroPLoP 的論文集裡，我們沒有找到以架構重建或相依分析為主題的 pattern 論文。主題上最接近的有：EuroPLoP 2022 的 Service Mesh Patterns [Maia and Correia 2022]（service mesh 與 sidecar 這組 pattern 的定義出處；本文的執行期觀測來自 service mesh）；AsianPLoP 2025 的 Patterns for Log Dissemination in Cloud-Native Environments [Ho et al. 2025]；AsianPLoP 2026 的工作坊論文 Patterns for AI-Assisted Document Maintenance and Traceability [Su et al. 2026]（接近 ④）；以及 PLoP 2023 的 A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT [White et al. 2023]（和 ④ 方向互補）。

11 討論與限制

適用範圍。這個 language 適用於：系統由多個獨立部署的元件組成；相依圖要從多個來源拼出來；有人會拿這張圖做決策。只有單一來源的工具（例如純粹從追蹤產生的服務圖）不需要 ①②；只畫給人瀏覽、不拿來做決策的圖，④ 的必要性也較低。

效度威脅。第一，兩個條件證明「別人也這樣做」，但不能證明「照這個順序建構一定順利」；pattern 之間的銜接順序是我們從文獻與文件歸納的，沒有實證研究驗證過。第二，業界證據來自官方文件，文件寫的不一定等於產品的實際行為；其中 ④ 的 Dynatrace CoPilot 證據來自官方部落格。第三，LLM 的行為會變：④ 的必要性建立在 LLM 會重述與填空之上；若未來的模型不再如此，這個 pattern 要處理的 forces 會變弱，但它的解法（不讓模型重述事實）在那種情況下也不會造成損害。

12 結論

重建微服務相依圖的第一步（讀部署描述檔、抽取程式碼呼叫點、看執行期服務圖），學界與業界的做法已經很一致；本文整理的是之後的四個做法：先補齊流量再把來源合成一張圖；標出實測與推測的差別；請人確認規則判斷不了的名稱對應；讓程式從同一份模型寫事實、LLM 只寫解釋，並把沒有資料和零分開。每一個 pattern 都有論文與業界工具兩方面的實例；證據較弱的地方，我們在第 9 節說明了弱在哪裡。

致謝（ACKNOWLEDGMENTS）

感謝本文的 shepherd〔姓名〕在 shepherding 過程中的建議，以及 AsianPLoP 2027 Writers' Workshop〔組別〕的成員〔姓名〕的討論與回饋。

參考文獻（REFERENCES）

Abdullah Al Maruf, Alexander Bakhtin, Tomas Cerny, and Davide Taibi. 2022. Using Microservice Telemetry Data for System Dynamic Analysis. In *Proceedings of the 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 29–38. https://doi.org/10.1109/SOSE55356.2022.00010

Nuha Alshuqayran, Nour Ali, and Roger Evans. 2018. Towards Micro Service Architecture Recovery: An Empirical Study. In *Proceedings of the 2018 IEEE International Conference on Software Architecture (ICSA)*. https://doi.org/10.1109/ICSA.2018.00014

Amazon Web Services. *AWS X-Ray Concepts*. AWS X-Ray Developer Guide. Retrieved October 13, 2026 from https://docs.aws.amazon.com/xray/latest/devguide/xray-concepts.html

Amazon Web Services. *View Your Application Topology and Monitor Operational Health with the CloudWatch Application Map*. Amazon CloudWatch User Guide. Retrieved October 13, 2026 from https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/ServiceMap.html

Backstage. *Backstage Software Catalog*. Backstage documentation. Retrieved October 13, 2026 from https://backstage.io/docs/features/software-catalog/

Backstage. *Well-known Relations between Catalog Entities*. Backstage documentation. Retrieved October 13, 2026 from https://backstage.io/docs/features/software-catalog/well-known-relations

Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, and Davide Taibi. 2024. Tools Reconstructing Microservice Architecture: A Systematic Mapping Study. In *Software Architecture. ECSA 2023 Tracks, Workshops, and Doctoral Symposium* (LNCS, Vol. 14590). Springer, 3–18. https://doi.org/10.1007/978-3-031-66326-0_1

Andreas Bauer, Martin Leucker, and Christian Schallhart. 2011. Runtime Verification for LTL and TLTL. *ACM Transactions on Software Engineering and Methodology* 20, 4 (2011), Article 14. https://doi.org/10.1145/2000799.2000800

Vincent Bushong, Dipta Das, and Tomas Cerny. 2022. Reconstructing the Holistic Architecture of Microservice Systems using Static Analysis. In *Proceedings of the International Conference on Cloud Computing and Services Science (CLOSER 2022)*. SciTePress, 149–157. https://doi.org/10.5220/0011032100003200

Tomas Cerny, Amr S. Abdelfattah, Vincent Bushong, Abdullah Al Maruf, and Davide Taibi. 2022. Microservice Architecture Reconstruction and Visualization Techniques: A Review. In *Proceedings of the 2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. 39–48. https://doi.org/10.1109/SOSE55356.2022.00011

Bas Cornelissen, Andy Zaidman, Arie van Deursen, Leon Moonen, and Rainer Koschke. 2009. A Systematic Survey of Program Comprehension through Dynamic Analysis. *IEEE Transactions on Software Engineering* 35, 5 (2009), 684–702. https://doi.org/10.1109/TSE.2009.28

Datadog. *Configure Monitors*. Datadog documentation. Retrieved October 13, 2026 from https://docs.datadoghq.com/monitors/configuration/

Datadog. *Inferred Services*. Datadog documentation. Retrieved October 13, 2026 from https://docs.datadoghq.com/tracing/services/inferred_services/

Datadog. *Service Map*. Datadog documentation. Retrieved October 13, 2026 from https://docs.datadoghq.com/tracing/services/services_map/

Datadog. *Service Remapping Rules*. Datadog documentation. Retrieved October 13, 2026 from https://docs.datadoghq.com/tracing/services/service_remapping_rules/

Dynatrace. *Custom Topology Model*. Dynatrace documentation. Retrieved October 13, 2026 from https://docs.dynatrace.com/docs/ingest-from/extend-dynatrace/extend-topology

Michalis Famelis, Rick Salay, and Marsha Chechik. 2012. Partial Models: Towards Modeling and Reasoning with Uncertainty. In *Proceedings of the 34th International Conference on Software Engineering (ICSE 2012)*. 573–583. https://doi.org/10.1109/ICSE.2012.6227159

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided Language Models. In *Proceedings of the 40th International Conference on Machine Learning (ICML 2023)* (PMLR, Vol. 202). 10764–10799.

Google Cloud Platform. *Online Boutique*. GitHub repository. Retrieved October 13, 2026 from https://github.com/GoogleCloudPlatform/microservices-demo

Grafana Labs. *No Data and Error States*. Grafana documentation. Retrieved October 13, 2026 from https://grafana.com/docs/grafana/latest/alerting/fundamentals/alert-rule-evaluation/nodata-and-error-states/

Grafana Labs. *Service Graphs*. Grafana Tempo documentation. Retrieved October 13, 2026 from https://grafana.com/docs/tempo/latest/metrics-from-traces/service_graphs/

Olena Grinenko, Andrii Gizun, Artem Morhun, and Andrii Bekh. 2026. Methods for Reconstructing Microservice Architectures: A Thematic Literature Review. In *Proceedings of the Fifth International Workshop on Cyber Hygiene (WCH 2026)* (CEUR Workshop Proceedings, Vol. 4278). 131–146. https://ceur-ws.org/Vol-4278/paper10.pdf

Hua Guo, Jeff Huang, and David H. Laidlaw. 2015. Representing Uncertainty in Graph Edges: An Evaluation of Paired Visual Variables. *IEEE Transactions on Visualization and Computer Graphics* 21, 10 (2015), 1173–1186. https://doi.org/10.1109/TVCG.2015.2424872

Evan Ho, Yi-Hsiu Chiu, and Chun-Feng Liao. 2025. Patterns for Log Dissemination in Cloud-Native Environments. In *Proceedings of the 11th Asian Conference on Pattern Languages of Programs, People, and Practices (AsianPLoP 2025)*. The Hillside Group, 15 pages. https://plopcon.org/proceedings/asianplop/2025/papers/sa1.pdf（論文集 DOI https://doi.org/10.64346/AsianPLoP2025）

Istio. *Visualizing Your Mesh*. Istio documentation. Retrieved October 13, 2026 from https://istio.io/latest/docs/tasks/observability/kiali/

K8sGPT. *K8sGPT*. GitHub repository. Retrieved October 13, 2026 from https://github.com/k8sgpt-ai/k8sgpt

Kiali. *Graph (FAQ)*. Kiali documentation. Retrieved October 13, 2026 from https://kiali.io/docs/faq/graph/

Venla Liljas, Matteo Esposito, Valentina Lenarduzzi, and Davide Taibi. 2026. Can Agents Reconstruct Microservice Architecture? In *Proceedings of the International Symposium on Empirical Software Engineering and Measurement (ESEM 2026)* (LIPIcs, Vol. 394). 62:1–62:13. https://doi.org/10.4230/LIPIcs.ESEM.2026.62

João Tiago Duarte Maia and Filipe Figueiredo Correia. 2022. Service Mesh Patterns. In *Proceedings of the 27th European Conference on Pattern Languages of Programs (EuroPLoP '22)*. ACM, Article 2, 12 pages. https://doi.org/10.1145/3551902.3551962

Benjamin Mayer and Rainer Weinreich. 2018. An Approach to Extract the Architecture of Microservice-Based Software Systems. In *Proceedings of the 2018 IEEE Symposium on Service-Oriented System Engineering (SOSE)*. 21–30. https://doi.org/10.1109/SOSE.2018.00012

Gail C. Murphy, David Notkin, and Kevin Sullivan. 1995. Software Reflexion Models: Bridging the Gap between Source and High-Level Models. In *Proceedings of the 3rd ACM SIGSOFT Symposium on Foundations of Software Engineering (SIGSOFT '95)*. ACM, 18–28. https://doi.org/10.1145/222124.222136

Gail C. Murphy, David Notkin, and Kevin J. Sullivan. 2001. Software Reflexion Models: Bridging the Gap between Design and Implementation. *IEEE Transactions on Software Engineering* 27, 4 (2001), 364–380. https://doi.org/10.1109/32.917525

Mohammad Imranur Rahman, Sebastiano Panichella, and Davide Taibi. 2019. A Curated Dataset of Microservices-Based Systems. In *SSSME 2019* (CEUR Workshop Proceedings, Vol. 2520). 1–9. https://arxiv.org/abs/1909.03249

Alois Reitbauer and Stefan Greifeneder. 2023. Dynatrace Expands Davis AI with Davis CoPilot, Pioneering the First Hypermodal AI Platform for Unified Observability and Security. Dynatrace blog (July 25, 2023). Retrieved October 13, 2026 from https://www.dynatrace.com/news/blog/hypermodal-ai-dynatrace-expands-davis-ai-with-davis-copilot/

Sunita Sarawagi and Anuradha Bhamidipaty. 2002. Interactive Deduplication using Active Learning. In *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '02)*. 269–278. https://doi.org/10.1145/775047.775087

Simon Schneider and Riccardo Scandariato. 2023. Automatic Extraction of Security-Rich Dataflow Diagrams for Microservice Applications Written in Java. *Journal of Systems and Software* 202 (2023), 111722. https://doi.org/10.1016/j.jss.2023.111722

Simon Schneider, Alexander Bakhtin, Xiaozhou Li, Jacopo Soldani, Antonio Brogi, Tomas Cerny, Riccardo Scandariato, and Davide Taibi. 2025. Comparison of Static Analysis Architecture Recovery Tools for Microservice Applications. *Empirical Software Engineering* 30, 5 (2025), Article 128. https://doi.org/10.1007/s10664-025-10686-2

ServiceNow. *Manage CI Connections for a Single Service Using Suggestions*. ServiceNow product documentation. Retrieved October 13, 2026 from https://www.servicenow.com/docs/r/Tm19yM2Ui3~uN9XFBacppg/u~Zf40ssZOZ1CiZacOJOVQ

ServiceNow. *Traffic-based Discovery in Service Mapping*. ServiceNow product documentation. Retrieved October 13, 2026 from https://www.servicenow.com/docs/r/Tm19yM2Ui3~uN9XFBacppg/N~7rM~61XplreSWjhnedAg

Jacopo Soldani, Javad Khalili, and Antonio Brogi. 2022. Offline Mining of Microservice-based Architectures. In *Proceedings of the International Conference on Cloud Computing and Services Science (CLOSER 2022)*. SciTePress, 63–73. https://doi.org/10.5220/0011061000003200

Marko Sterbentz, Cameron Barrie, Shubham Shahi, Abhratanu Dutta, Donna Hooshmand, Harper Pack, and Kristian J. Hammond. 2024. Satyrn: A

Platform for Analytics Augmented Generation. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP 2024)*. Association for Computational Linguistics, 6360–6385. https://doi.org/10.18653/v1/2024.emnlp-main.365

Dong-Yi Su, Yuan-Kai Liu, Wei-Hua Chien, Yan-Cheng Lin, and Shang-Pin Ma. 2026. Patterns for AI-Assisted Document Maintenance and Traceability. Writers' workshop paper, Asian Conference on Pattern Languages of Programs (AsianPLoP 2026). Retrieved October 14, 2026 from https://plopcon.org/asianplop2026/program/

vFunction. *Dynamic and Static Analysis*. vFunction knowledge base. Retrieved October 13, 2026 from https://kb.vfunction.com/analysis/architectural-observability/dynamic-static-analysis

Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. 2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. In *Proceedings of the 30th Conference on Pattern Languages of Programs (PLoP 2023)*. The Hillside Group. https://doi.org/10.64346/PLoP2023p05